CMAN :=

Given data, N=32, FP32 = 4 bytes, DRAM bandwidth = 320GB/s
compute = 10TFLOPS.

Task 1 :=

N=32 times, so each B[K][J] accessed 32 times

A access $N^3 = 32^3 = 32,768$

B access $N^3 = 32^3 = 32,768$

Total elements access $= 2N^3 = 2 \times (32)^3 = 65,536$

Total DRAM units $= 2N^3 \times 4 = 65,536 \times 4$

$= 262,144$ bytes $\simeq 256KB$

$\boxed{\text{Total DRAM traffic} = 256KB}$

Task 2 :=

Tile size (T=8)

Number of tiles per dimension $= N/T = \frac{32}{8} = 4$

A Tile loads the each A&B tiles used N/T times
(once per tile column of B)

A loads $= \left(\frac{N}{T}\right)^3 \times T^2 = (4)^3 \times 8^2 = 64 \times 64 = 4096$ elements

B title also same = 4096 elements

Total DRAM traffic $= 2 \times 4096 \times 4 = 32,768$ bytes

$= 32KB$

$\boxed{\text{Total DRAM traffic} = 32KB}$

Task 3 :=

Traffic ratio :=

Naive traffic = 262,144 bytes
Tiled traffic = 32,768 bytes

$$\text{Traffic ratio} = \frac{\text{Naive}}{\text{Tiled}} = \frac{262,144}{32,768} = 8 \ (T)$$

Tiled traffic = Naive traffic ÷ N

= 262,144 ÷ 32

Tiled traffic = 8,192 bytes

$$\text{Ratio} = \frac{262,144}{8,192} = 32 = N$$

- In the naive traffic, we reload matrix B from memory 32 times because every row of A needs to read all of B again. Tiling fixes this by keeping B in fast memory and reusing it, so we only load B once. That is why native method uses 32 times more memory traffic exactly N times more.

Task 4 :=

Bandwidth = 320GB/s
compute = 10TFLOPS
Total FLOPs = $2 \times N^3 = 2 \times 32^3 = 65,536$ FLOPs.

Naive case :=

t_memory = Traffic / BW

= 262144 / (320 GB/s)

$= 8.192 \times 10^{-7}$ s = 0.819 μs

t_compute = FLOPs/P

$= 65,536 / (1 \times 10^{13})$

$= 6.554 \times 10^{-9} s = 0.00655 \mu s$

t_memory $(0.819 \mu s) >>$ t_compute $(0.00655 \mu s)$

$\boxed{t_exec(naive) = 0.819 \mu s}$

Bottleneck : Memory-bound.

Tiled case (T=8) :=

t_Memory = Traffic/BW

$= 32,768 / (3.2 \times 10^{11})$

$= 1024 \times 10^{-7} s = 0.1024 \mu s$

t_compute = FLOPs/P

$= 65,536 / (1.0 \times 10^{13})$

$= 6.554 \times 10^{-9} s = 0.00655 \mu s$

t_memory $(0.1024 \mu s) >>$ t_compute $(0.00655 \mu s)$

$\boxed{t_exec(tiled) = 0.1024 \mu s}$

Bottleneck : Memory-bound